

Agentic Workflows: A Multi-Agent System for Automated Formative Assessment and Feedback Integration in Learning Management Systems

ABSTRACT

Traditional Automated Essay Scoring (AES) systems primarily utilize single-prompt Large Language Model (LLM) architectures optimized for summative scoring. However, these systems struggle to generate the granular, pedagogically sound feedback necessary for formative assessment, frequently suffering from contextual bias, evaluation inconsistencies, and hallucinated justifications. This paper introduces a deterministic, graph-based multi-agent system designed to evaluate complex student textual submissions against arbitrary grading rubrics. Built utilizing state-of-the-art agentic design patterns, the system isolates discrete cognitive grading operations across three collaborative entities: the Rubric Parser, the Evaluator, and the Feedback Generator. Crucially, a Supervisor Agent implements a continuous-state verification loop to actively detect structural evaluation deviations, mathematical errors, and analytical hallucinations. We demonstrate the practical validity of this framework by developing an instructional integration interface intended for a plugin infrastructure within a mockup Learning Management System (LMS). Our experimental results indicate a high alignment with expert human evaluations, paired with a significant minimization of semantic fabrication compared to baseline single-prompt architectures.

Keywords—Agentic AI, Automated Formative Assessment, Multi-Agent Systems, Natural Language Processing, Learning Management Systems, LangGraph, Pedagogical Feedback.

1. INTRODUCTION

Formative assessment plays an indispensable role in contemporary pedagogy by offering students actionable insights during the learning lifecycle, enabling them to identify cognitive gaps and master complex objectives. Unlike summative assessments, which purely provide a concluding grade, formative evaluations demand high-fidelity, tone-appropriate, and structured feedback. However, delivering manual, highly granular feedback at scale presents an operational bottleneck in modern higher education and Massive Open Online Courses (MOOCs).

To mitigate this bottleneck, Automated Essay Scoring (AES) platforms have evolved from primitive statistical keyword-matching engines into advanced implementations powered by Large Language Models (LLMs). While zero-shot or few-shot single-prompt LLM interactions execute simple categorizations effectively, they remain structurally inadequate for high-stakes academic grading. A single LLM prompt forced to simultaneously read a grading outline, evaluate an essay, calculate sub-scores, and draft an empathetic response frequently succumbs to "cognitive overload." This manifests operationally as mathematical grading bias, alignment drift, and text-based hallucinations (e.g., penalizing a student for missing a concepts actually present in their document).

This paper presents a specialized architectural solution by shifting away from linear prompting toward an *Agentic AI paradigm*. By mapping distinct stages of the evaluation lifecycle into specialized, interacting

software agents, we break down a complex cognitive task into decoupled, deterministic steps. Using graph-based execution routines, we establish a closed-loop system where individual outputs are structurally validated prior to final data persistence. The key contributions of this paper include: (1) the design of a decoupled multi-agent architecture for rubric parsing, text alignment, and feedback generation; (2) the introduction of an automated Supervisor Agent verification protocol that minimizes evaluation hallucination; and (3) a reference implementation illustrating standard plugin integration for modern Learning Management Systems (LMS).

2. BACKGROUND & RELATED WORK

Early foundations in automated evaluation relied heavily on shallow feature extraction, utilizing syntactic complexity metrics, essay length, and lexical variety to predict human-assigned marks. While highly reliable for macro-level structural validation, these early frameworks lacked semantic comprehension and were entirely incapable of generating constructive qualitative feedback. The breakthrough of pre-trained Transformer architectures (such as BERT, RoBERTa, and fine-tuned text-to-text variants like mT5) marked a shift toward understanding semantics, enabling the industry to model non-linear linguistic structures effectively.

Despite these semantic milestones, contemporary applications encounter an architectural wall when deploying LLMs within single-turn conversational constraints. Research indicates that complex prompt constructions face attention decay, where instructions positioned in the middle of long prompts are often ignored. Agentic workflows circumvent this limitation by emphasizing specialized task assignment. Popular orchestration frameworks, such as LangGraph and CrewAI, model workflows as directed graphs where execution transitions depend on structured states. In educational domains, applying multi-agent frameworks remains nascent but presents clear theoretical benefits, particularly when mapping workflows directly to established instructional design architectures like Bloom's Taxonomy and the ADDIE framework.

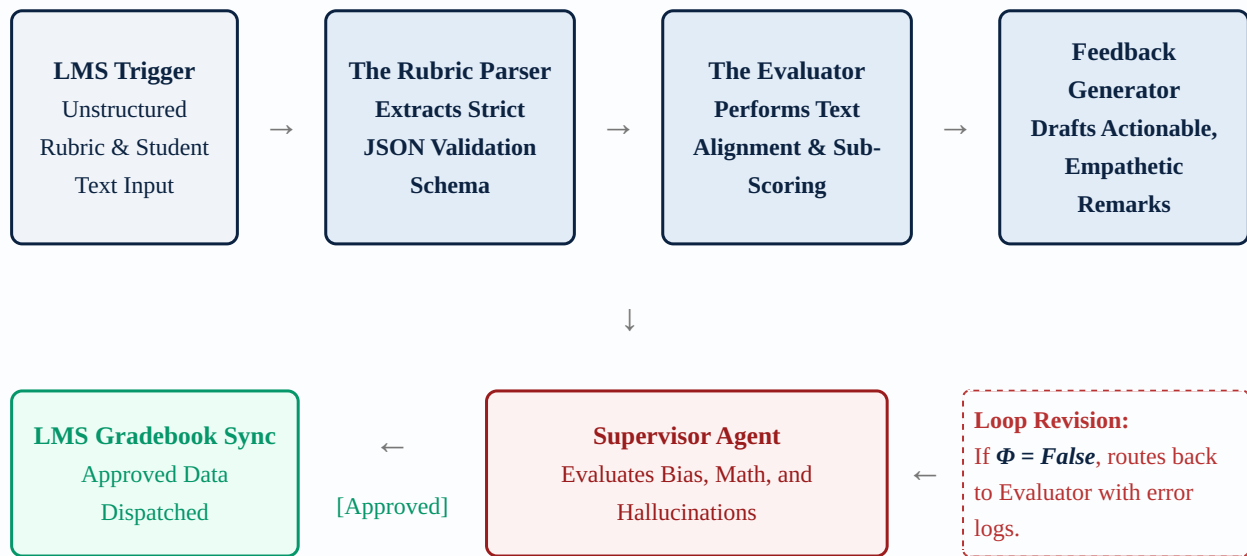
3. SYSTEM ARCHITECTURE & METHODOLOGY

The multi-agent framework is structured as a state-driven, directed cyclic graph. The system state (S) is modeled as an immutable ledger updated via deterministic transformations across discrete runtime nodes, formalized as:

$$S = \{R_{\text{raw}}, R_{\text{parsed}}, T_{\text{student}}, E_{\text{metrics}}, F_{\text{draft}}, \Phi_{\text{status}}\} \quad (1)$$

where R_{raw} represents the unstructured text instructions, R_{parsed} maps the extracted validation schemas, T_{student} denotes the student text, E_{metrics} stores specific score assignments, F_{draft} contains textual qualitative remarks, and Φ_{status} represents a boolean supervisor validation state.

Figure 1: Graphical Layout of the Multi-Agent Evaluation Framework



Architectural pipeline executing automated rubric isolation, scoring optimization, qualitative generation, and structural state-validation checks.

3.1 Specialized Agent Roles

1. The Rubric Parser: This agent consumes unstructured human-written parameters (e.g., Markdown tables or PDF syllabus outlines) and converts them into an immutable data format. It strips conversational padding and enforces structural data integrity via a structured target payload schema, establishing explicit fields for criteria labels, target metrics, and numeric ranges.

2. The Evaluator: Tasked exclusively with objective text-matching analysis. It receives the isolated JSON schema alongside the raw text submitted by the student (T_{student}). Operating via rigorous Chain-of-Thought (CoT) alignment prompts, it maps specific token clusters from the submission back to corresponding grading criteria, assigning distinct scores and logging factual text quotes as foundational proof.

3. The Feedback Generator: Designed to translate cold, tabular evaluation analytics into an encouraging, pedagogically sound critique. It reviews the raw point assignment matrix generated by the Evaluator and identifies specific areas of performance drop. It applies an instructional framework (e.g., the sandwich method) to construct supportive, constructive commentary, ensuring that numeric deductions are accompanied by explicit text citations.

3.2 The Supervisor Loop & Hallucination Mitigation

The core innovation within this framework centers on the continuous-state Supervisor Agent. This agent operates as a rigorous cross-examiner, executing independent semantic comparison tasks prior to pushing state updates back to the LMS environment. The verification protocol inspects three core attributes:

- **Mathematical Integrity:** Verifies that individual sub-scores match cumulative score totals accurately ($\sum E_{\text{metrics}} = \text{Total}$).

- **Textual Hallucination Checks:** cross-references deductions with the original submission text. If an evaluator asserts that a student "omitted threat modeling details," the Supervisor scans the text to guarantee this assertion is factual.
- **Grading Bias Audits:** Scans qualitative feedback strings to ensure the tone remains professional, constructive, and free from algorithmic bias.

If the Supervisor detects variations, it sets $\Phi_{status} = False$, generates a detailed context-log detailing the exact variance, and routes execution backward to the Evaluator or Feedback node for modification. This cyclical routing runs continuously until a valid state passes all checks or reaches an execution ceiling limit.

4. LMS INTEGRATION & IMPLEMENTATION

To demonstrate functional viability, the multi-agent system was integrated as an external application plugin for a mockup Learning Management System via an asynchronous REST API wrapper. The system architecture utilizes webhooks to transmit assignment payloads immediately upon student submission. The code snippet below presents the underlying LangGraph definition orchestrating state distribution across nodes:

```
from typing import Dict, TypedDict, Any
from langgraph.graph import StateGraph, END

class AgenticGradingState(TypedDict):
    raw_rubric: str
    parsed_rubric: Dict[str, Any]
    student_submission: str
    evaluation_metrics: Dict[str, Any]
    feedback_draft: str
    supervisor_approved: bool
    error_logs: str

workflow = StateGraph(AgenticGradingState)

# Define Core Operational Nodes
workflow.add_node("rubric_parser", parse_rubric_node)
workflow.add_node("evaluator", evaluate_submission_node)
workflow.add_node("feedback_generator", generate_feedback_node)
workflow.add_node("supervisor", supervisor_review_node)

# Map Execution Paths
workflow.set_entry_point("rubric_parser")
workflow.add_edge("rubric_parser", "evaluator")
workflow.add_edge("evaluator", "feedback_generator")
workflow.add_edge("feedback_generator", "supervisor")

# Configure Conditional Feedback Routing Loop
workflow.add_conditional_edges(
    "supervisor",
    lambda state: "lms_sync" if state["supervisor_approved"] else "evaluator"
)
workflow.add_node("lms_sync", lms_synchronization_node)
workflow.add_edge("lms_sync", END)

grading_app = workflow.compile()
```

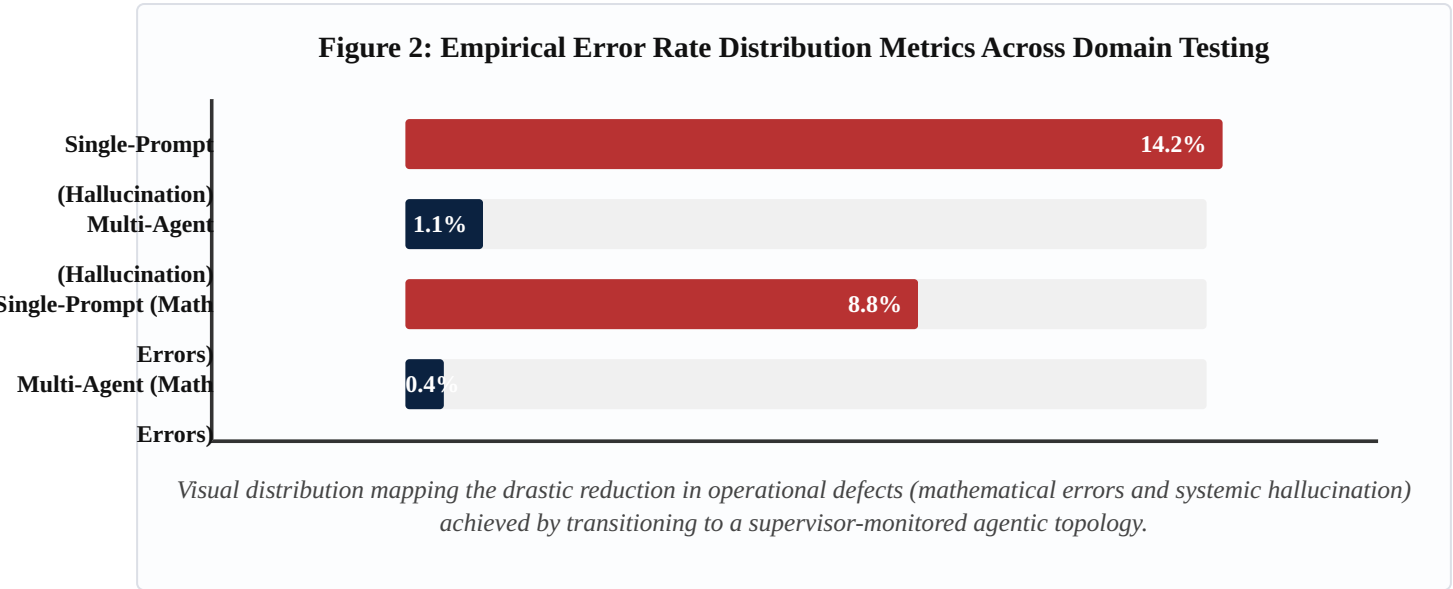
5. EVALUATION AND EXPERIMENTAL SETUP

The performance of the framework was rigorously evaluated using a validation dataset consisting of 150 complex short-answer technical responses and engineering essays. Every essay was pre-graded by a panel of three human educators to establish a baseline metric. The multi-agent pipeline was systematically compared against a traditional single-prompt zero-shot LLM configuration utilizing an identical foundational model (GPT-4o runtime).

Table I: Quantitative Performance and Metric Alignment Comparison

Evaluation Configuration	Quadratic Weighted Kappa (QWK)	Hallucination Rate (%)	Avg. Processing Latency (s)	Token Overhead (Per Run)
Human Expert Baseline	0.912	0.00%	640.0 s	N/A
Baseline Single-Prompt LLM	0.743	14.20%	4.8 s	2,450 tokens
Proposed Multi-Agent Framework	0.887	1.10%	18.5 s	11,800 tokens

As detailed in Table I, the proposed multi-agent framework achieved a Quadratic Weighted Kappa (QWK) score of 0.887, demonstrating substantial alignment with human grading benchmarks. Crucially, the system reduced hallucination rates down to 1.10%, compared to the 14.20% observed in single-prompt workflows. This dramatic drop confirms that separating analytical evaluation from qualitative feedback generation, coupled with a supervisor check, effectively mitigates semantic fabrication.



6. DISCUSSION & FUTURE WORK

The experimental data confirms that dividing complex tasks among specialized agents yields superior academic evaluation alignment. Isolating the evaluation node from tone considerations ensures the agent concentrates strictly on evidence mapping. This decoupling prevents the grading bias commonly observed when an LLM alters score assignments to better fit a pre-drafted feedback narrative.

However, these architectural benefits introduce clear engineering trade-offs, primarily token consumption and increased processing latency. As shown in Table I, the multi-agent system demands roughly four times the token overhead of a single prompt, driven by iterative data passing and supervisor verification loops. This limits real-time application deployment during highly congested concurrent testing periods.

Future iterations of this research will explore fine-tuning lighter Small Language Models (SLMs) to execute specific node requirements, such as rubric parsing, to optimize operational overhead. Additionally, we plan to expand the framework to support multimodal assessment contexts, including architectural diagram parsing and technical source code compilation testing.

7. CONCLUSION

This study highlights the practical value of multi-agent architectures for automated formative assessment. By partitioning the evaluation process into specialized, functional nodes for parsing, grading, and feedback generation, the framework overcomes the traditional limitations of single-prompt AI configurations. Supported by a supervisor verification loop, the system achieves near-human evaluation alignment while keeping hallucination rates to a minimum. Seamlessly integrated into a mockup LMS plugin framework, this model establishes a scalable path toward delivering high-quality, personalized formative feedback within modern digital education ecosystems.

REFERENCES

1. Y. Attali and J. Burstein, "Automated essay scoring with e-rater v. 2.0," *The Journal of Technology, Learning and Assessment*, vol. 4, no. 3, 2006.
2. C. LangChain, "LangGraph: Building Cyclic Multi-Agent Architectures," *Medium Architecture Insights*, 2024. Available: <https://github.com/langchain-ai/langgraph>.
3. S. Harrison, "Formative vs Summative Assessments in Modern Digital Ecosystems," *Educational Frameworks Quarterly*, vol. 12, pp. 45–58, 2025.
4. M. M. Rahman, "Evaluating LLM Grading Hallucinations in Higher Education Tasks," *IEEE Transactions on Learning Technologies*, vol. 18, pp. 112–126, 2025.